PROJECT EXPLANATION FOR EVALUATORS

# CloudSentinel

*AI-Powered Adaptive Zero Trust for Multi-Cloud Applications*

*same user + same moment + different resource sensitivity = different access decision*

| Item | Detail |
|---|---|
| What it is | A working hackathon prototype of an Adaptive Zero Trust control plane. |
| Core claim | A signed-in user is never permanently trusted. Access is decided per application, using current risk and resource sensitivity. |
| Version | V1.1 — FastAPI backend, React dashboard, Isolation Forest behavior input, optional MongoDB. |
| Demo login | developer01 / CloudDemo123! |
| How to use this PDF | Read it section by section while showing the live dashboard. It explains what was built, how decisions are made, and what is simulated. |

# 1. What to tell the evaluator first

CloudSentinel is not a login page that later trusts the session. It is a control plane that continuously evaluates whether this user, on this device, with this behaviour, should be allowed to touch this application right now. The frontend only displays decisions. The FastAPI policy engine is the authority.

The product story is blast-radius reduction. After an account-compromise simulation, the same user is not blocked everywhere. Lower-sensitivity applications can remain usable. High-sensitivity applications are restricted or denied. That is the point of Adaptive Zero Trust in this prototype.

# 2. Problem

Cloud organisations still treat authentication as a one-time gate. Once a password or token is accepted, Email, HR records, customer databases, and production infrastructure often share the same implicit trust. If credentials leak, the blast radius is the entire session.

- Static IAM does not re-evaluate device posture, location, or behaviour after login.
- A global block of the user is too blunt: it also stops low-risk work.
- Security tools that only output a mysterious ML score cannot explain why access was denied.
- Multi-cloud applications have very different sensitivity, but many demos still use one allow/deny bit per user.

# 3. Solution

CloudSentinel inserts a Zero Trust Gateway between the user and simulated cloud applications. For every access check it gathers identity, latest telemetry, endpoint posture, security tags, behavioural anomaly, and a transparent 0–100 risk score. The Adaptive Policy Engine then maps user risk against resource sensitivity and returns one of: ALLOW, MFA_REQUIRED, READ_ONLY, DENY, ISOLATE, plus a human-readable reason and the policy rule name.

# 4. End-to-end decision flow

This is the pipeline implemented in backend/security/zero_trust_gateway.py. API route handlers stay thin. The gateway orchestrates the modules so policy logic does not live in FastAPI or in the React UI.

| Step | Module | What happens |
|---|---|---|
| 1 | Identity | JWT login. User, role, and organisation are loaded. Passwords are bcrypt hashed. |
| 2 | Telemetry | Latest activity: requests/min, download MB, failed logins, unique apps, access frequency, login hour, location. |
| 3 | Posture | Device trust, OS compliance, antivirus, MFA, location vs registered device. |
| 4 | Tags | Signals become tags such as TRUSTED_DEVICE, NEW_DEVICE, DATA_EXFILTRATION. Obsolete tags are deactivated. |
| 5 | Anomaly (AI) | scikit-learn Isolation Forest scores behaviour. This is one input, not the whole decision. |
| 6 | Risk engine | score = identity + posture + behavior + context, clamped 0–100, with reasons. |
| 7 | Policy engine | Risk level × resource sensitivity, then stricter device and action rules. |
| 8 | Audit | Access request and security events are stored for Incidents and Audit Logs. |

*Table 1. Zero Trust decision pipeline.*

# 5. Architecture and what was built

The repository is modular on purpose: telemetry, posture, tags, anomaly, risk, policy, gateway, API, and UI are separate. SQLite is the default demo database. MongoDB Atlas is optional (DATA_STORE=mongodb). The React dashboard polls backend state; it does not compute allow/deny locally.

### 5.1 Backend

- FastAPI application version 1.1.0 with CORS, health endpoints, auto-seed of demo data.
- SQLAlchemy models: organisations, users, devices, applications, tags, telemetry, risk scores, access requests, events, simulation state.
- JWT authentication (POST /api/auth/login, GET /api/auth/me).
- Admin user lifecycle, device register / heartbeat / posture ingest.
- Authoritative access APIs: POST /api/access/check and GET /api/users/{id}/access-matrix.
- Deterministic simulation: POST /api/simulation/attack and POST /api/simulation/reset.

### 5.2 Frontend (what you show on screen)

- Overview — risk gauge, posture tags, access matrix, live events, SIMULATE / RESET.
- How It Works — 12-stage live pipeline. Best page for explanation.
- Security Posture — per-user tags, behaviour, AI assessment, matrix.
- Organization / Users / Devices — tenant, admin user management, endpoint inventory.
- Resources — simulated AWS/Azure applications and sensitivity.
- Incidents and Audit Logs — timeline, containment, blast radius, policy events.

## 6. Risk engine and Isolation Forest

Risk is explainable arithmetic, not a hidden neural net. Identity risk (max 20) rises with failed logins. Posture risk comes from device and compliance. Context risk (max 15) rises if location differs from the registered device or the login hour is outside 06:00–20:00. Behaviour risk comes from the Isolation Forest (and a heuristic fallback if the model file is missing).

The model in ml/model.pkl is trained on a small deterministic healthy baseline (random_state=42, contamination=0.15). Features: requests_per_minute, data_download_mb, failed_logins, unique_applications, access_frequency, login_hour. Healthy demo telemetry is about 20 requests/min and 50 MB. Attack telemetry is 185 requests/min, 875 MB, 4 failed logins, location Mumbai, hour 02:00.

Levels: LOW below 30, MEDIUM below 60, HIGH below 80, CRITICAL at 80 and above. If the evaluator asks “is this AI?”, the accurate answer is: Isolation Forest detects unusual behaviour; the policy engine still decides access using risk, tags, device, action, and resource sensitivity.

## 7. Policy matrix — the main demo claim

POLICY_MATRIX in backend/security/policy_engine.py is the base mapping. Additional rules can only make the decision stricter: untrusted device on a high/critical resource, critical tags such as DATA_EXFILTRATION, and WRITE/DELETE/ADMIN actions. Administrators are not globally blocked; they receive step-up MFA when risk or the action is elevated.

| User risk \ Resource | LOW | MEDIUM | HIGH | CRITICAL |
|---|---|---|---|---|
| LOW | ALLOW | ALLOW | ALLOW | MFA_REQUIRED |
| MEDIUM | ALLOW | ALLOW | MFA_REQUIRED | MFA_REQUIRED |
| HIGH | ALLOW | MFA_REQUIRED | READ_ONLY | DENY |

| User risk \ Resource | LOW | MEDIUM | HIGH | CRITICAL |
|---|---|---|---|---|
| CRITICAL | MFA_REQUIRED | MFA_REQUIRED | READ_ONLY | DENY |

*Table 2. Base policy: risk level against resource sensitivity.*

### Seeded applications

| Application | Cloud | Sensitivity | Meaning |
|---|---|---|---|
| Email | Azure | 20 (LOW) | Collaboration mailbox. Often remains reachable after compromise. |
| HR Portal | Azure | 40 (MEDIUM) | Employee records. May require MFA as risk rises. |
| Customer Database | AWS | 75 (HIGH) | Production customer data. Restricted or denied when risk is high. |
| Production Kubernetes Cluster | AWS | 95 (CRITICAL) | Infrastructure control plane. Denied from a high-risk context. |

*Table 3. Demo applications. Sensitivity is an integer; policy uses LOW/MEDIUM/HIGH/CRITICAL bands.*

## 8. Live demo script (use this while clicking)

Start backend (uvicorn) and frontend (npm run dev). Open http://localhost:5173. Login as developer01 / CloudDemo123!.

- Healthy state: show LOW risk, trusted Windows device in Pune, MFA verified, Email allowed. Even here, a critical resource can already require stronger control. Access is per application, not one global switch.
- Click SIMULATE ACCOUNT COMPROMISE. Say clearly: this injects deterministic telemetry. It does not run malware or scan the laptop.
- Narrate the staged events: new device, location Pune → Mumbai, failed logins, request spike, 875 MB transfer, behavioural anomaly, risk recalculated, policies re-evaluated.
- Open the access matrix. Point out that Email can stay ALLOW or MFA while Customer Database and Kubernetes go DENY. That is blast-radius reduction.
- Click Customer Database. The modal shows risk, sensitivity, factors, policy_rule, and reason. This is the “why was I blocked?” answer.
- Open Incidents and Audit Logs. Every stage is auditable.
- Click RESET DEMO. Risk returns to LOW, healthy tags return, matrix returns to baseline. Repeatable without reseeding the whole database.

## 9. Technology stack

| Layer | Choice |
|---|---|
| API | Python, FastAPI, Pydantic, Uvicorn |
| Data | SQLAlchemy + SQLite (default). Optional PyMongo / MongoDB Atlas. |
| Auth | JWT, bcrypt password hashing |
| AI / ML | scikit-learn Isolation Forest on six behaviour features |
| UI | React, Vite, Tailwind CSS, Axios, Recharts, Lucide |
| Tests | pytest, API smoke scripts, Playwright/Chrome smoke |

*Table 4. Implemented stack.*

## 10. V1.1 control-plane foundation

Beyond the original demo pipeline, V1.1 adds organisation/tenant context, admin user APIs and UI, device registration, heartbeat, posture submission, telemetry source labels (SIMULATION, AGENT, CONTROL_PLANE), MongoDB health, and a clean Mongo seed path. Device APIs are shaped like a future Windows endpoint agent: register, heartbeat, posture. That agent is not shipped yet.

## 11. Honest boundaries — say this before you are asked

This is a controlled prototype. Cloud resources are simulated application metadata, not live AWS/Azure/GCP accounts. Endpoint threat signals are written as database/telemetry state. MFA_REQUIRED and ISOLATE are policy decisions, not a real MFA provider or network isolation. Isolation Forest uses demo telemetry. There is no real EDR, packet capture, host scanning, or cloud IAM write-back.

That honesty is a strength in evaluation: the engineering that does exist is real — modular policy, explainable risk, resource-specific decisions, audit trail, repeatable simulation.

## 12. Likely evaluator questions

| Question | Short answer |
|---|---|
| Is the UI faking decisions? | No. It calls /api/access/check and /access-matrix. Policy lives in Python. |
| Why not block the whole user? | Zero Trust here means least privilege per resource. Contain crown jewels; keep low-risk tools if policy allows. |
| Is Isolation Forest the product? | No. It feeds behaviour_risk. Posture, identity, action, and sensitivity still change the outcome. |
| Do you need MongoDB? | No. SQLite is the demo default. MongoDB is optional persistence. |
| Is this production? | No. Next steps: Windows agent, real telemetry, enforcement adapters, cloud IAM connectors. |

*Table 5. Prepared answers.*

## 13. How to run (if they ask)

From the project root: python -m pip install -r requirements.txt ; python -m backend.data.seed ; python -m ml.train ; python -m uvicorn backend.main:app --reload. In frontend: npm install ; npm run dev. Health: http://127.0.0.1:8000/api/health and OpenAPI at /docs.

## 14. Closing line

*CloudSentinel continuously decides which application this user may touch right now, explains the decision, restricts only what is sensitive after compromise, and can reset to a healthy baseline. That is the prototype. That is what to demonstrate.*